

Estruturas de Dados

Tema: análise experimental do tempo de execução de algoritmos de ordenação

Comparação de Desempenho de Algoritmos de Ordenação

1. Objetivo da atividade

O objetivo deste exercício é implementar, executar e comparar o desempenho de cinco algoritmos clássicos de ordenação: Bubble Sort, Insertion Sort, Selection Sort, Heap Sort e Quick Sort. Cada algoritmo deverá ordenar exatamente os mesmos conjuntos de dados, permitindo uma comparação justa entre os resultados obtidos.

Ao final da atividade, o aluno deverá montar uma tabela com os tempos de execução e gerar um gráfico de linhas no Microsoft Excel, demonstrando o comportamento de cada algoritmo à medida que o tamanho da entrada aumenta.

2. Descrição geral da atividade

O aluno deverá criar seis arquivos de entrada contendo letras em ordem aleatória. Cada arquivo representará um conjunto de dados com uma quantidade diferente de elementos. As letras poderão ser minúsculas e maiúsculas, por exemplo: a, b, c, d, ..., z, A, B, C, D, ..., Z. Como a quantidade de elementos será maior que o total de letras disponíveis no alfabeto, as letras poderão se repetir.

Cada arquivo deverá ser submetido aos cinco algoritmos de ordenação. Para cada execução, deverá ser medido o tempo necessário para ordenar os dados. A leitura do arquivo não deverá ser contabilizada no tempo de ordenação.

3. Arquivos de entrada

Devem ser gerados seis arquivos de entrada, conforme a tabela a seguir:

Arquivo	Quantidade de elementos
entrada10.txt	10 letras
entrada100.txt	100 letras
entrada1000.txt	1.000 letras
entrada10000.txt	10.000 letras
entrada100000.txt	100.000 letras
entrada1000000.txt	1.000.000 letras

Exemplo simplificado do conteúdo do arquivo entrada10.txt:

d, A, b, Z, e, C, a, M, x, B

Atenção: todos os algoritmos deverão receber os mesmos dados de entrada. Portanto, antes de executar cada algoritmo, o programa deve carregar novamente os dados originais do arquivo ou criar uma cópia do vetor original. Isso evita que um algoritmo receba dados já ordenados por outro algoritmo.

Estruturas de Dados

Tema: análise experimental do tempo de execução de algoritmos de ordenação

4. Algoritmos que deverão ser implementados

4.1. Bubble Sort

Compara pares de elementos vizinhos e realiza trocas quando estão fora de ordem. Esse processo é repetido até que todos os elementos estejam ordenados.

4.2. Insertion Sort

Constrói a ordenação gradualmente, inserindo cada elemento na posição correta em relação aos elementos anteriores.

4.3. Selection Sort

Seleciona, a cada iteração, o menor elemento da parte não ordenada do vetor e o coloca na posição correta.

4.4. Heap Sort

Utiliza a estrutura de dados heap para organizar os elementos e realizar a ordenação de forma eficiente.

4.5. Quick Sort

Utiliza a estratégia de divisão e conquista. Escolhe um pivô, particiona os elementos em dois grupos e ordena recursivamente cada parte.

5. Medição do tempo de execução

Para cada algoritmo, deverá ser medido somente o tempo gasto no processo de ordenação. A leitura dos arquivos, a geração das entradas e a gravação de resultados não devem fazer parte da medição do tempo de ordenação.

Procedimento recomendado:

1. Ler o conteúdo do arquivo de entrada.
2. Armazenar os dados em um vetor ou lista.
3. Criar uma cópia do vetor original para cada algoritmo.
4. Iniciar a contagem do tempo imediatamente antes da chamada do algoritmo de ordenação.
5. Executar o algoritmo de ordenação.
6. Finalizar a contagem do tempo imediatamente após o término da ordenação.
7. Registrar o tempo obtido na tabela de resultados.

Exemplo conceitual para medição:

```
inicio = tempo_atual()
executar_algoritmo_de_ordenacao(vetor)
fim = tempo_atual()

tempo_execucao = fim - inicio
```

6. Tabela de resultados

Após executar todos os algoritmos para todos os tamanhos de entrada, deverá ser montada uma tabela com o seguinte formato:

Estruturas de Dados

Tema: análise experimental do tempo de execução de algoritmos de ordenação

Algoritmo	10 valores	100 valores	1.000 valores	10.000 valores	100.000 valores	1.000.000 valores
Bubble Sort						
Insertion Sort						
Selection Sort						
Heap Sort						
Quick Sort						

Cada célula deverá conter o tempo de execução do algoritmo para ordenar o respectivo conjunto de dados. A unidade de tempo poderá ser segundos, milissegundos ou microssegundos, desde que seja a mesma em toda a tabela.

7. Construção do gráfico no Excel

Com a tabela preenchida, o aluno deverá criar um gráfico de linhas no Excel para comparar visualmente o comportamento dos cinco algoritmos.

- Eixo X: quantidade de elementos ordenados: 10, 100, 1.000, 10.000, 100.000 e 1.000.000.
- Eixo Y: tempo de execução.
- Cada algoritmo deverá ser representado por uma linha no gráfico.
- O gráfico deverá conter título, legenda e identificação adequada dos eixos.

Título sugerido para o gráfico:

Comparação de Desempenho dos Algoritmos de Ordenação

8. Análise dos resultados

Após gerar a tabela e o gráfico, responda às questões a seguir:

- Qual algoritmo apresentou melhor desempenho para pequenas entradas, considerando 10, 100 e 1.000 elementos?
- Qual algoritmo apresentou melhor desempenho para grandes entradas, considerando 100.000 e 1.000.000 elementos?
- Quais algoritmos tiveram crescimento mais acentuado no tempo de execução?
- O comportamento observado confirma a complexidade teórica dos algoritmos?
- Algum algoritmo ficou inviável para entradas muito grandes? Explique.

Complexidades médias esperadas:

Algoritmo	Complexidade média esperada
Bubble Sort	$O(n^2)$
Insertion Sort	$O(n^2)$
Selection Sort	$O(n^2)$
Heap Sort	$O(n \log n)$
Quick Sort	$O(n \log n)$

Estruturas de Dados

Tema: análise experimental do tempo de execução de algoritmos de ordenação

9. Observações importantes

Os algoritmos Bubble Sort, Insertion Sort e Selection Sort possuem complexidade quadrática e tendem a apresentar desempenho ruim em entradas muito grandes. Portanto, ao ordenar 100.000 ou 1.000.000 de elementos, esses algoritmos podem demorar muito ou se tornar impraticáveis, dependendo do computador utilizado.

Caso algum algoritmo demore tempo excessivo, registre na tabela uma observação, como "Não concluído" ou "Tempo excedido", e comente essa situação na análise final.

10. Entregáveis

1. Código-fonte do programa contendo os cinco algoritmos de ordenação.
2. Seis arquivos de entrada: entrada10.txt, entrada100.txt, entrada1000.txt, entrada10000.txt, entrada100000.txt e entrada1000000.txt.
3. Tabela com os tempos de execução.
4. Gráfico de linhas gerado no Excel.
5. Breve análise dos resultados obtidos.

11. Resultado esperado

Ao final da atividade, espera-se que o aluno compreenda, na prática, que algoritmos diferentes podem produzir o mesmo resultado final, isto é, a ordenação dos dados, mas apresentar desempenhos muito distintos.

Também se espera que o aluno perceba que algoritmos simples, como Bubble Sort, Insertion Sort e Selection Sort, podem funcionar bem para pequenas entradas, mas tendem a ser pouco eficientes para grandes volumes de dados. Por outro lado, algoritmos como Heap Sort e Quick Sort geralmente apresentam desempenho superior em entradas maiores, devido à sua complexidade média mais eficiente.